

Otimização de balanceadores de carga de sistemas de distribuição de conteúdo por meio de monitoramento sistemático de memória

Pedro Faria Fernandes

Universidade do Estado de Santa Catarina

10 de novembro de 2025

Sumário I

Introdução

Contexto

Evolução do Consumo de Vídeo

- Popularização do consumo de conteúdo em vídeo (televisão);
- Internet e *streaming* transformaram o hábito para **sob demanda**;
- Crescimento **exponencial** de dados de vídeo na rede;
- Pressão sem precedentes sobre a infraestrutura global, causada pela demanda;

Desafios de Otimização

- Maximizar a Qualidade de Experiência (QoE) para usuários;
- Considerando que QoE é responsabilidade compartilhada entre provedores de serviço e ISPs (*Internet Service Providers*), o foco deste trabalho será a otimização da qualidade no **contexto do provedor de serviço**;

Problema e Justificativa

- Sistemas de distribuição de conteúdo $\approx$ múltiplos servidores, um balanceador de carga;
- O uso de memória (principal e secundária) é correlacionado com o desempenho de servidores de distribuição de conteúdo e é um cenário comum em CDNs (*Content Delivery Networks*) com **servidores de borda** (*Edge Servers* ou *Cache Servers*) ?;
- Há uma lacuna em estratégias convencionais de balanceamento de carga que ignoram o estado dos recursos de memória ou consideram outros recursos computacionais em conjunto;

Observação: Os termos "uso de memória" e "consumo de memória", no contexto deste trabalho, correspondem às métricas de **consumo de memória principal** e **taxa de entrada e saída (I/O) de memória secundária**.

Objetivo

Objetivo do trabalho

- O objetivo do trabalho é seguir pela abordagem da consideração exclusiva do uso de memória de modo a avaliar uma estratégia de otimização da escolha do servidor em balanceadores de carga para sistemas de distribuição de conteúdo.
- O objetivo será atingido por meio da implementação, amparada pelo estabelecimento de uma arquitetura de simulação para testes, de um algoritmo de balanceamento de carga e posterior análise comparativa.

Trabalhos relacionados

Trabalhos Relacionados I

- **NGINX:** Sistema popular que utiliza algoritmos como *Round-Robin* que podem levar a desequilíbrios, pois não monitoram o **uso de memória** dos servidores em tempo real para tomar decisões ?.
- **HAProxy:** É reconhecido pela alta eficiência e baixo consumo de recursos, mas também carece de um mecanismo nativo de monitoramento de memória para otimizar a distribuição de tráfego ?.
- **Docker Swarm:** Embora um estudo tenha explorado adicionar a **influência da memória** em seu balanceador, o foco foi exclusivo do ecossistema Docker, sem aplicação em servidores de distribuição de conteúdo ?.
- **Algoritmo Dinâmico (Cloud):** Uma solução foi proposta para lidar com *Flash Crowds* em aplicações na nuvem usando múltiplas métricas, mas sem especificidade para o gargalo de memória em CDNs ?.
- **Revisão de Parâmetros:** Uma revisão de algoritmos dinâmicos mostrou que, embora alguns usem **memória e disco** como parâmetros, nenhum os utiliza como fator principal ou exclusivo de avaliação de desempenho ?.

Trabalhos Relacionados II

- **Streaming de Vídeo (iSCON):** A estratégia iSCON foca em replicar conteúdo popular para balancear a carga, mas utiliza a **largura de banda** como principal indicador, ignorando a pressão sobre a memória ?.
- **Servidores de Vídeo Distribuídos:** Um trabalho propôs uma solução de duas fases que mede a carga pelo **número de streams ativas**, uma abstração que não considera gargalos de recursos de baixo nível como a memória ?.

Fundamentação Teórica

Content Delivery Networks (CDNs)

CDNs são infraestruturas cruciais para entregar conteúdo rápido, confiável e eficiente, distribuindo-o em servidores geograficamente dispersos.

- São **sistemas distribuídos de computadores**, administrados no nível da aplicação, que encaminham requisições para o servidor mais próximo do usuário, reduzindo latência e sobrecarga ?;
- **Arquitetura**: Servidores de origem (conteúdo original) e servidores de borda (cópias redundantes) em PoPs (*Points of Presence*);
- **Cache**: Conteúdo é servido do PoP mais próximo, otimizando a distância física e reduzindo a carga na rede global;
- **Roteamento**: Utilizam DNS para direcionar usuários ao servidor ideal com base na localização geográfica;
- **Balanceamento de Carga**: Distribuem requisições para evitar sobrecarga em PoPs, usando localização geográfica e técnicas como *anycast* ?;

Balancedores de Carga

Balancedores de carga são cruciais em sistemas distribuídos, incluindo CDNs.

Função e Importância

- Distribuem eficientemente cargas de trabalho (requisições, tráfego) entre múltiplos servidores e são essenciais para a **escalabilidade horizontal**, preferencial à vertical por flexibilidade e custo ?;
- Melhoram o desempenho, a escalabilidade, a confiabilidade e a disponibilidade (ex: redirecionamento em caso de falha);
- A eficácia do sistema distribuído depende da sofisticação do balanceamento de carga;
- Categorias principais: **Nível de aplicação** e **nível de rede**; ?;

MPEG-DASH

HTTP (Hypertext Transfer Protocol) Streaming é padrão para multimídia, levando ao **DASH** (*Dynamic Adaptive Streaming over HTTP*) ?.

DASH: Arquitetura e Componentes

- **Vantagem:** Abordagem *stateless* do HTTP é escalável, contrastando com RTP ?;
- **Orientado ao Cliente:** Servidor disponibiliza; cliente gerencia a inteligência da sessão ?;
- **MPD (Media Presentation Description):** Manifesto XML com URLs e versões do conteúdo (Períodos, Adaptation Sets, Representações) ?;
- **Segmentos:** Conteúdo dividido em blocos, requisitados via HTTP GET, facilmente *cacheáveis* por CDNs ?;
- **Content Steering:** Cliente decide *de onde* buscar o conteúdo via múltiplas URLs base no MPD ?;

Proposta

Objetivo do Algoritmo

- O objetivo principal é definir o método de escolha de um servidor para cada nova requisição;
- A natureza do algoritmo é **probabilística**;
 - A chance de um servidor ser escolhido é baseada em uma probabilidade calculada dinamicamente (de acordo com a Equação ??);
- A probabilidade é influenciada principalmente pelo Consumo de **Memória RAM** e pela taxa de entrada e saída de **Disco (I/O)**;
- O funcionamento do algoritmo é **dinâmico**;
 - Os servidores enviam dados de monitoramento (carga) periodicamente para o balanceador, que atualiza os parâmetros para as próximas requisições;

Cálculo da probabilidade

A probabilidade (P_i) de uma requisição ser enviada para um servidor i é baseada em uma versão adaptada do algoritmo *Basic Load Interpretation* (*Basic LI*) ?. A equação utilizada corresponde exatamente à equação determinada pelo *Basic LI*:

$$P_i = \frac{\frac{L_{tot} + arrive_T}{n} - L_i}{arrive_T} \quad (1)$$

P_i : Probabilidade de escolha do servidor i ;

n : Número total de servidores disponíveis;

L_i : Carga individual reportada pelo servidor i ;

L_{tot} : Somatório de todas as cargas reportadas ($\sum L_i$);

$arrive_T$: Número total de requisições esperadas no sistema inteiro para o tempo médio (T) das informações de carga ($\sum R_i$);

Adaptação do Parâmetro de Carga (L_i)

No contexto deste trabalho, o termo "carga" (L_i) é adaptado (de tamanho de fila de requisições para uso de memória) para refletir o desempenho de um servidor de conteúdo. A carga é calculada a partir de métricas específicas por meio da Equação ??.

$$L_i = CM \cdot LM_i + CD \cdot LD_i \quad (2)$$

LM_i : Consumo de **memória** normalizado do servidor i (em %);

LD_i : Carga de **disco** (I/O) normalizada do servidor i (em %);

CM, CD : Coeficientes dinâmicos que representam a importância geral da memória e do disco no sistema, respectivamente;

Cálculo dos Coeficientes Dinâmicos (CM e CD)

Os coeficientes dão mais peso ao recurso que está mais sobrecarregado no sistema como um todo. Eles são calculados a partir das taxas de uso gerais de memória (TM) e disco (TD). A propriedade $CM + CD = 1$ garante que a importância seja distribuída de forma proporcional entre os dois recursos.

- Taxa de Memória: $TM = \sum_{i=0}^n \frac{LM_i}{n}$
- Taxa de Disco: $TD = \sum_{i=0}^n \frac{LD_i}{n}$

$$CM = \frac{TM}{TM + TD} \quad (3)$$

$$CD = \frac{TD}{TM + TD} \quad (4)$$

O algoritmo é executado a cada nova requisição que chega ao balanceador e sua execução é determinada conforme o pseudo-código a seguir.

Pseudocódigo

Algorithm Seleção de Servidor por Requisição

Gatilho: Chegada de uma nova requisição R .

Require:

S : Lista de servidores com métricas L_i , R_i e T_i atualizadas.

Variáveis globais: L_{tot}, n .

Ensure:

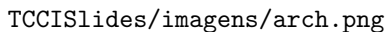
Requisição R encaminhada para um servidor S_i .

```

1: procedure SELECIONARSERVIDOR( $R, S, L_{tot}, n$ )
2:    $arrive_T \leftarrow$  CalcularQtdRequisicoesEsperadas( $S$ )
3:    $P_{list} \leftarrow []$  ▷ Lista de probabilidades
4:   for all  $S_i, L_i$  in  $S$  do
5:      $P_i \leftarrow ((L_{tot} + arrive_T)/n - L_i)/arrive_T$  ▷ Eq. ??
6:     Adicionar ( $S_i, P_i$ ) a  $P_{list}$ 
7:    $s_{escolhido} \leftarrow$  SelecaoAleatoriaComPesos( $P_{list}$ )
8:   Encaminhar( $R, s_{escolhido}$ )

```

Arquitetura geral do sistema



TCCISlides/imagens/arch.png

Fonte: Elaborado pelo autor (2025)

Metodologia de Validação e Métricas

- **Estratégia de Validação:**

- Análise comparativa rigorosa executada em cenários de **alta carga**;
- O mesmo nível de sobrecarga é usado como linha de base para todos os testes, garantindo uma comparação justa;

- **Definição de Sobrecarga:**

- Ponto definido empiricamente quando a Experiência do Usuário (QoE) se degrada, com **travamentos visíveis** (*stalls*) no player de vídeo;

- **Métricas de Avaliação (QoE):**

- Número total de travamentos (*stalls*) do vídeo;
- Duração média de cada travamento;
- Tempo total de ociosidade do serviço (soma das pausas);

Critérios de Comparação e Cenários

- **Algoritmos Comparados:**

- **Proposto:** Baseado na interpretação de dados de carga (mesmo que obsoletos);
- **Clássicos:** *Round-Robin* e Seleção Aleatória, executados na mesma aplicação para uma comparação justa;

- **Cenários de Execução:**

- Balanceamento com **Round-Robin**;
- Balanceamento com **Seleção Aleatória**;
- Balanceamento proposto (intervalo de atualização de **6 segundos**);
- Balanceamento proposto (intervalo de atualização de **60 segundos**);
- Balanceamento proposto (intervalo de atualização de **600 segundos**);

- **Hipótese Central:**

- O algoritmo proposto será superior aos clássicos, mesmo com dados de carga consideravelmente desatualizados (60s e 600s);

Considerações parciais

Proposta Central do Trabalho

- **O Problema Identificado:**

- Estratégias clássicas (*Round-Robin*, Aleatória) ignoram o estado real dos recursos dos servidores, impactando a Qualidade de Experiência (QoE);
- A **memória** (principal e secundária) é um fator de gargalo crítico em servidores que entregam conteúdo a partir de arquivos;

- **A Solução Proposta:**

- Um algoritmo de balanceamento **probabilístico** que integra o monitoramento de recursos como fator decisório;
- Adapta o conceito de *Load Interpretation* para considerar não apenas o consumo de recursos, mas também a **idade da informação**;

Próximos Passos

De modo resumido, os próximos passos são definidos conforme a lista a seguir:

1. Configuração e testes do sistema publicar/assinar (*broker* MQTT);
2. Implementação e configuração inicial do servidor MPEG-DASH;
3. Implementação do balanceador de carga;
4. Criação da aplicação cliente geradora de sobrecarga no sistema;
5. Configuração do *player* simples no navegador;
6. Execução dos testes de qualidade com o *player* simples;
7. Elaboração dos gráficos e descrição dos resultados finais;

Cronograma

Atividades	Meses					
	Jul	Ago	Set	Out	Nov	Dez
1						
2						
3						
4						
5						
6						
7						

Referências

Referências I